

Task 2 — Submission Report

From Tables to Notebooks

Student	Ziad Suleyman
Course	MLOPS Training — 2026/2027
Trainer	Trainer Qafza
Problem	Predict late vs on-time delivery
Notebooks	6, run in order from a clean kernel
Artifacts	13 files + 12 saved figures
Final result	ROC-AUC 0.711 on the held-out test
Report date	6 September 2026

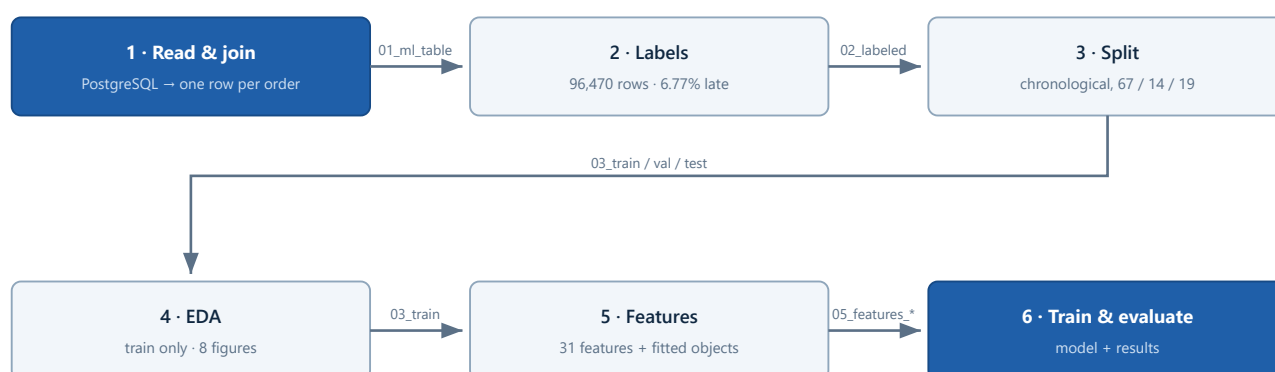
Every number and console output in this report was produced by
executing the six notebooks end to end.

0. “Done when” — self-check against the task sheet

Criterion (from the task PDF)	Status	Evidence
Six notebooks, each doing one job, running in order from a clean start	Done	\$1 — all six re-executed top to bottom on a fresh kernel; execution counts run 1..n in order
Every notebook saves its artifacts, and reads the artifacts of the step before it	Done	\$1 — the artifact chain; 13 files under <code>artifacts/</code>
You can explain why you split the data the way you did	Done	\$4 — the split is chronological, and the reason is a measured number, not an argument
You have a first model result compared against a simple baseline	Done	\$7 — four baselines, then the selected model, then one look at the test set

1. The pipeline

Each notebook does one job, reads the artifact written by the step before it, and writes its own for the step after it. No notebook needs an earlier one to be running.



#	Notebook	Artifacts written
1	<code>01_read_and_join.ipynb</code>	<code>01_ml_table.parquet</code>
2	<code>02_labels.ipynb</code>	<code>02_labeled.parquet</code>
3	<code>03_split.ipynb</code>	<code>03_train.parquet</code> · <code>03_val.parquet</code> · <code>03_test.parquet</code>
4	<code>04_eda.ipynb</code>	<code>04_findings.md</code> + 8 figures
5	<code>05_features.ipynb</code>	<code>05_features_{train,val,test}.parquet</code> · <code>05_transformers.joblib</code> · <code>05_feature_list.json</code>
6	<code>06_train_evaluate.ipynb</code>	<code>06_model.joblib</code> · <code>06_results.md</code> + 4 figures

Artifacts are **parquet** rather than CSV so column types — timestamps in particular — survive the handoff and the next notebook never has to re-parse them.

2. Notebook 1 — one row per order

The task's rule for this step is *aggregate before you join*. Three tables hold several rows per order, so joining them directly would multiply the table instead of describing it.

EVIDENCE 1 — which tables need aggregating

tbl	rows	distinct_orders	
orders	99441	99441	<- already one row per order
order_items	112650	98666	
order_payments	103886	99440	
order_reviews	99224	98673	

Two consequences drove the whole notebook:

- **775 orders have no rows in `order_items`** (98,666 of 99,441). An `INNER JOIN` would have dropped them silently — no error, no warning. Every join is therefore a `LEFT JOIN` starting from `orders`.
- **`geolocation` holds 1,000,163 rows for 19,015 zip prefixes** (~53 points each). It is collapsed to one *median* coordinate per prefix before joining — the median rather than the mean because the table contains points outside Brazil entirely.

Orders with more than one seller

1,278 orders involve several sellers, which makes “the seller's state” ambiguous. The notebook defines the **main line** of an order as its highest-priced line, and takes the seller, category, state and zip from it — a deterministic, explainable rule — while keeping `n_sellers` as a separate column so the multi-seller fact itself is not lost.

EVIDENCE 2 — the merge, with the row count checked at every step

```
+ items      -> 99,441 rows (was 99,441)
+ main_line  -> 99,441 rows (was 99,441)
+ products   -> 99,441 rows (was 99,441)
+ payments   -> 99,441 rows (was 99,441)
After adding coordinates: 99,441 rows x 37 columns

OK 99,441 rows = 99,441 orders
OK order_id is unique
OK 37 columns
```

Every merge passes `validate="one_to_one"`, which raises immediately if a key is not unique on either side — a failed aggregation surfaces as an exception instead of as duplicated rows.

3. Notebook 2 — the label

The obvious definition is `delivered > estimated`. Before accepting it, the notebook asks one question about the estimate itself.

EVIDENCE 3 — is the promise an instant or a day?

```
Promised dates whose time is 00:00:00 : 96,470 out of 96,470
Share: 100.00%
```

Every promised date carries the time `00:00:00`. The promise is a **day**, and midnight is simply the absence of an hour — not a commitment to deliver at midnight. Comparing timestamps therefore marks an order delivered *on*

the promised day as late.

EVIDENCE 4 — how much that costs

```
1,292 orders on which the two definitions disagree
= 16.5% of the positive class under the literal definition

At what hour of the promised day did these orders arrive?
min      0.0
25%     14.0
50%     18.0    <- median arrival: 6 p.m. on the promised day
75%     20.0
max      23.0

is_late    (primary, calendar) : 6,534 late  (6.77%)
is_late_ts (alternative, literal) : 7,826 late  (8.11%)
```

Decision	Reasoning
Primary label is the calendar rule (6.77%)	The promise is a date. The 1,292 disputed orders arrive at a median hour of 18:00 — calling a 6 p.m. delivery on the promised day “late” injects 16.5% false positives into the class the model learns from.
The literal rule is kept as <code>is_late_ts</code>	Both columns ship in the artifact, so switching definitions is a one-line change.
Population = <code>delivered</code> and has a delivery date	An undelivered order is neither late nor on time. 6 canceled orders that carry a delivery date are excluded: the question is “will it arrive on time”, not “did it arrive then get canceled”.

EVIDENCE 5 — the label re-derived by an independent method and compared row by row

```
OK both methods agree on all 96,470 rows - the label is logically correct
```

Class balance: 6,534 late out of 96,470 — one late order for every 13.8 on time. The imbalance is moderate rather than severe: the minority is rare in percentage terms but numerous in absolute terms, so the difficulty lies in choosing the metric, not in finding examples.

4. Notebook 3 — why the split is chronological

This is the criterion the task asks to be explained. The notebook answers it with a measurement rather than an argument.

The usual argument does not apply here

Random splits are normally rejected because they leak customer history across the boundary. That was checked first:

EVIDENCE 6 — repeat customers

```
Distinct customers      : 93,350
Ordered more than once  : 2,801 (3.00%)
Ordered exactly once    : 90,549 (97.00%)
```

97% of customers ordered once. There is almost no customer history to leak, so this argument is weak in this dataset. The real reason lies elsewhere.

The real reason, as a number

The late rate is not stable over time — it ranges from under 1% to nearly 19% depending on the month. So the notebook measures how much signal the calendar alone carries: it uses *the late rate of an order's own month* as if it were a prediction, and scores it.

EVIDENCE 7 — the month, used as a predictor

ROC-AUC using the month's late rate alone as the prediction: **0.698**

Knowing **only the month** scores 0.698 — higher than the purchase-time baseline model measured in Task 1 (0.679). A random split places March 2018 (19% late) and June 2018 (1% late) in train *and* test simultaneously, letting the model memorise month-level base rates. That information does not exist in production, where the model predicts for a month that has not happened yet.

The split, and the boundaries

Split	Window	Orders	Share	Late
train	up to 2018-03-31	64,320	66.7%	7.95%
validation	2018-04-01 → 2018-05-31	13,547	14.0%	5.53%
test	from 2018-06-01	18,603	19.3%	3.61%

Three candidate cut points were compared before choosing. An earlier validation window would have given it a late rate roughly triple the test set's, making it a misleading proxy for exactly the thing it exists to predict. The chosen boundaries keep validation and test adjacent in time.

The differing late rates are deliberate. The task asks to preserve the label ratio across splits *if splitting randomly*; this split is chronological, so the rates are left alone. The shift from 7.95% to 3.61% is the phenomenon the model has to face honestly, not a defect to be corrected.

EVIDENCE 8 — verification, and one disclosed limitation

OK $64,320 + 13,547 + 18,603 = 96,470$

OK no `order_id` overlap between the splits

OK no time overlap - every split precedes the next

Orders in train delivered on/after the start of validation: **3,852 (5.99% of train)**

Median delivery duration: 10 days

The split is by *purchase* date, but a label is only known at *delivery*. 3,852 training orders (5.99%) were delivered after the validation window opened. The strict remedy — an embargo — would cost 6% of the training data for a small theoretical gain, so those rows were kept and the limitation is disclosed rather than hidden.

5. Notebook 4 — what the EDA decided

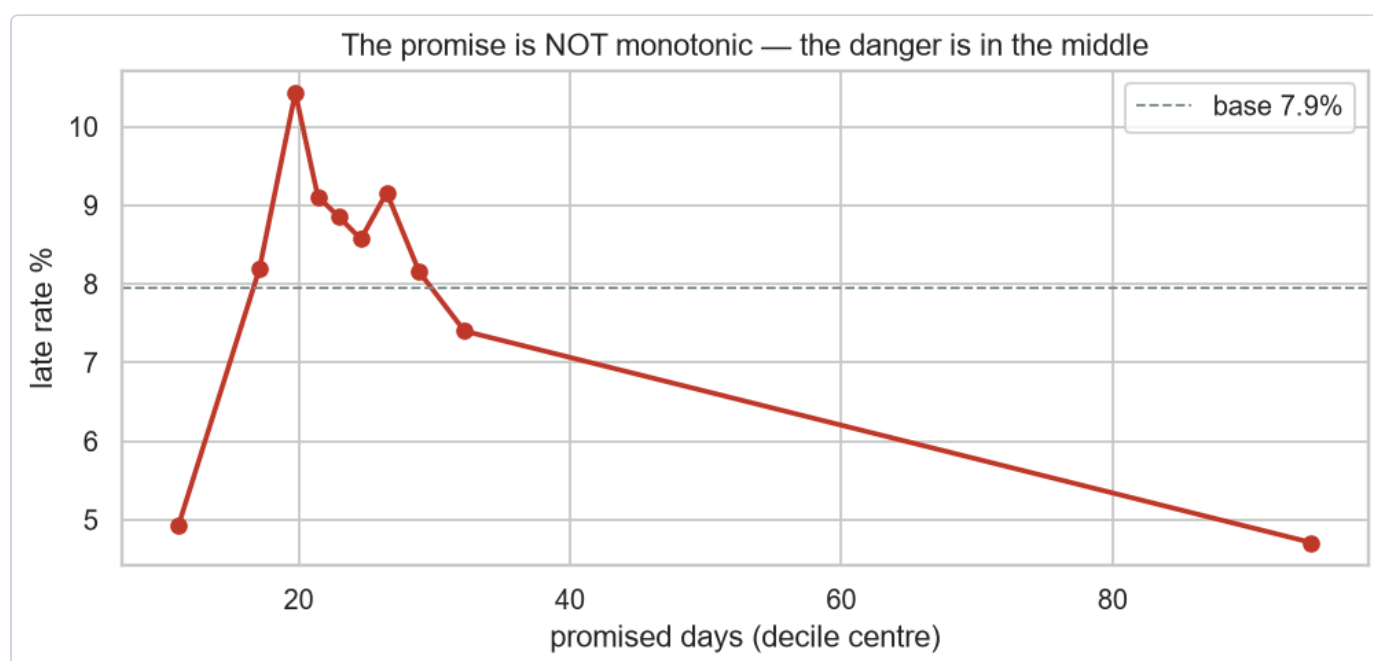
Run on `03_train.parquet` only. The goal was not charts but two written decisions: which features to build, and which model family suits the shape of the data.

The promise looks useless — and is not

EVIDENCE 9

AUC of `promised_days` alone: **0.487**
(0.5 means: no signal whatsoever)

AUC measures a *monotonic* relationship. The actual shape is an inverted U: short promises are safe, long promises are safe, and the danger sits in the middle.



Late rate across deciles of promised days — 4.9% at the short end, 10.4% in the middle, 4.7% at the long end

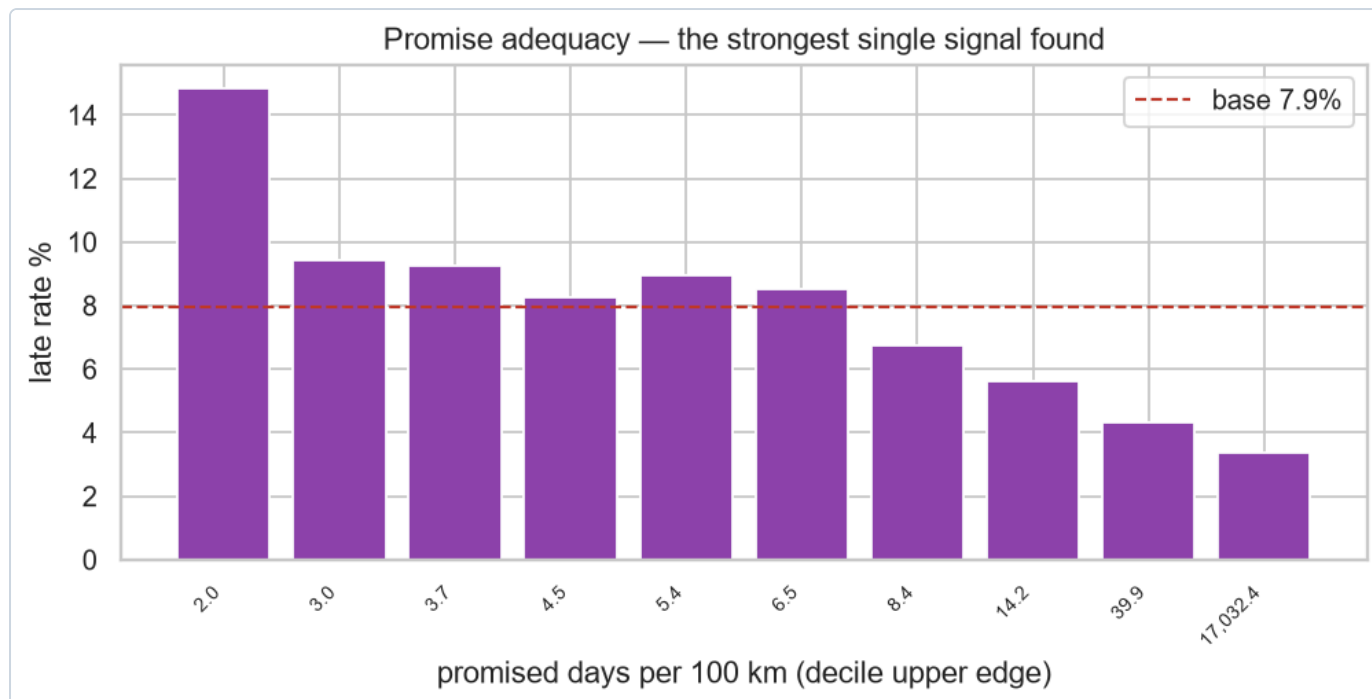
The correlation that explains it, and the feature it produces

EVIDENCE 10

Spearman correlation between the promise and the distance: **0.65**

AUC of `promise_per_100km`: 0.388 -> in the reversed direction: **0.612**
for comparison - distance alone: 0.589
promise alone : 0.487

The company already lengthens its promise as distance grows, so the two effects cancel when the promise is viewed alone. The informative quantity is therefore not *how many days were promised* but **how many days were promised per 100 km** — whether the promise is generous or ambitious relative to the shipment.



Promise adequacy — from 14.9% late at the most ambitious decile down to 3.4% at the most generous

The rest of the findings

Finding	Measurement
Distance dominates the raw columns	AUC 0.589, monotonic from 3.7% to 12.8% across deciles
Same state halves the risk	4.61% vs 9.66%
State matters a great deal	4.72% (PR) to 19.43% (MA)
Missing coordinates are themselves a signal	173 orders with no geolocation match are late 10.98% of the time vs 7.94% — a zip prefix absent from the reference table means a remote area. Kept as an indicator column rather than imputed away
Product attributes carry nothing	price, weight, volume, item count all between 0.49 and 0.53 — lateness is a route problem, not a parcel problem
Day of week and hour are flat	7.38% to 8.55% across weekdays
Month is strong but excluded	0.75% to 18.96%, but it is <i>March 2018</i> specifically, not “March” — it cannot generalise forward

6. Notebook 5 — features, and the objects that must be saved

31 features. The discipline this step enforces: every statistic is learned from the training split alone, and everything learned is saved. In production a single order arrives — a median cannot be computed from it.

Saved object	What it learned	What refitting it in production would break
<code>kept_categories</code>	which of the 72 product categories are not rare (44 kept, 28 grouped)	different categories collapse, so column meanings shift
<code>target_encoders</code>	smoothed late rate per state and category	an encoding built from data the model never saw
<code>imputer</code>	the median of every feature	gaps filled with different numbers than during training
<code>onehot_payment</code>	the payment values and their order	a different column order scatters the features
<code>scaler</code>	mean and standard deviation of every feature	a linear model reading meaningless inputs

Target encoding without leaking the label

Replacing a state with its historical late rate introduces two hazards, both handled explicitly:

- **Small categories produce noise.** A state with 12 orders and 2 late reads 16.7%. Smoothing pulls each estimate toward the overall rate with a strength of 50 pseudo-observations, so small categories barely move from the prior while large ones keep their own rate.
- **Rows would otherwise see their own label.** Encoding a training row with a mean computed from the training set includes that row's label in its own feature. Inside train the encoding is therefore computed **out of fold** — 5 folds, each encoded from the other four. Validation and test use the full-train mapping, which is exactly what production will use.

EVIDENCE 11 — verified reproducibility, not claimed reproducibility

```
OK 31 features, no leakage, no gaps, identical structure across splits
OK rebuilding from the loaded objects matches the notebook output exactly
-> the pipeline is genuinely reproducible, not just claimed to be
```

The final cell reloads `05_transformers.joblib` from disk, rebuilds a single test order through the loaded objects alone, and asserts the result matches what the notebook produced. This is the task's requirement that a production pipeline's output must match the notebook's — tested rather than assumed.

The leakage blacklist is a variable in the code, not a comment, and an assertion fails if any blocked column reaches the feature table. The scaler is fitted and saved but deliberately *not* applied: trees do not need it, and the stored tables stay readable.

7. Notebook 6 — results

The test file is not loaded until the final section, after every decision has been fixed.

Why accuracy is not the metric

EVIDENCE 12

The 'nothing is late' model:
 accuracy : **94.47%** <- a number that looks excellent
 late orders caught: **0** out of 749 <- and it is worthless

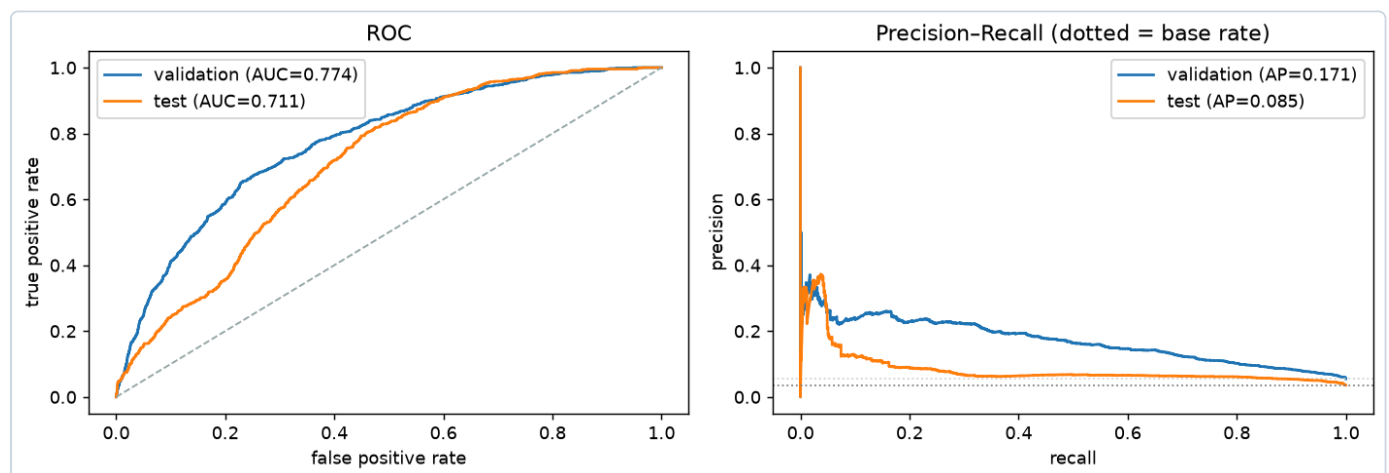
A model that predicts nothing is ever late scores 94.47%. The metrics used instead are PR-AUC (primary), ROC-AUC (comparable across splits with different base rates), and **recall at a fixed alert budget** — how many late orders are caught if the team can intervene on the riskiest 5%.

Baselines, then models — measured on validation

Model	ROC-AUC	PR-AUC
Predict nothing is late	—	—
B1 — <code>promise_per_100km</code> alone, no training at all	0.6953	0.1433
M1 — Random Forest	0.7303	0.1490
M2 — Gradient Boosting (depth 2)	0.7526	0.1709
Logistic regression — selected	0.7739	0.1709

The single look at the test set

Metric	validation	test
Orders	13,547	18,603
Late rate	5.53%	3.61%
ROC-AUC	0.7739	0.7107
PR-AUC	0.1709	0.0849
recall @ 5% alert budget	20.7%	15.0%
lift @ 5%	4.14x	3.01x



ROC and precision-recall on validation and test. The dotted lines mark each split's base rate

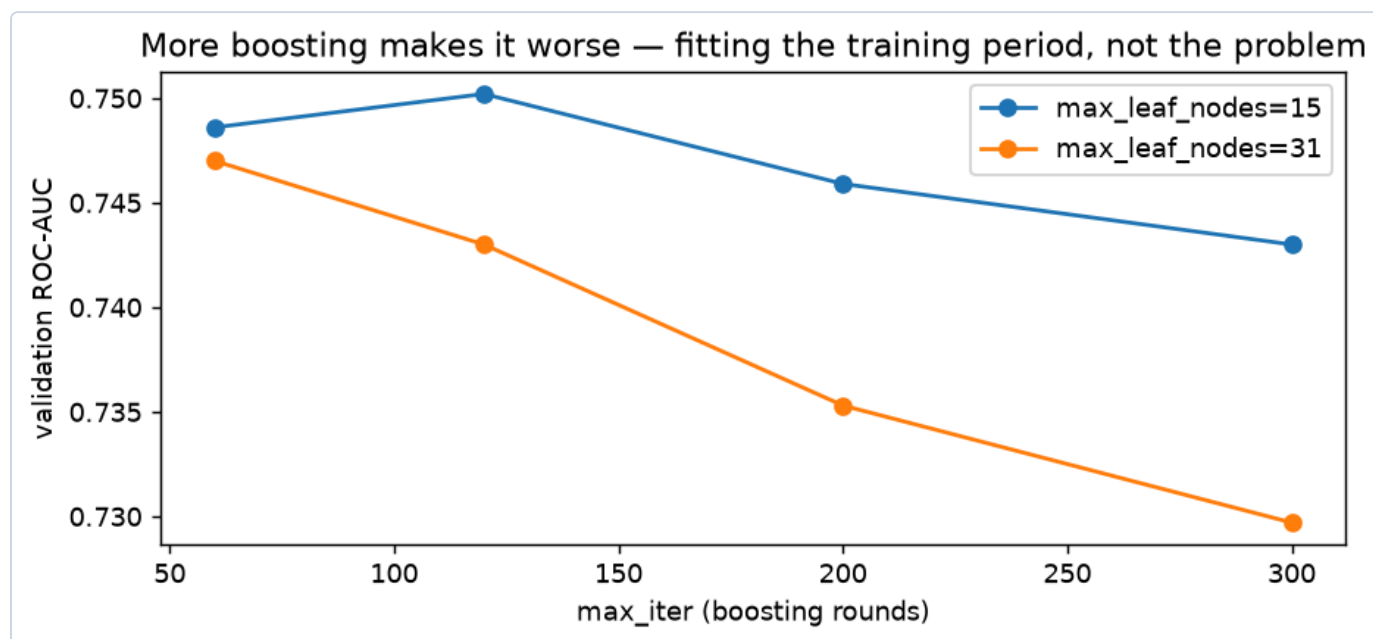
Operationally: intervening on the riskiest 5% of orders catches 15% of the late ones on data the model has never seen — three times what random targeting would find. The reference figure from Task 1 (purchase-time features, a different label and split) was ROC-AUC 0.679.

On the gap between 0.774 and 0.711: ROC-AUC is a ranking metric and is not affected by the base rate, so this is a real loss of generalisation rather than an artefact. Training ends in March 2018 while the test window runs June to August — the model is predicting three to five months past anything it has seen. The operational conclusion is periodic retraining rather than a single deployment.

8. Two measured results that contradicted the prediction

8.1 The trees lost

Notebook 4 predicted tree models would win, for three stated reasons: the non-monotonic promise curve, heavy skew, and real interactions between columns. Logistic regression beat them on both metrics. The evidence for why is visible in the tuning itself — boosting performance *degrades* as rounds increase, with everything else held constant:



Validation ROC-AUC against boosting rounds — more training makes it worse

That is not undertraining; it is the model memorising the training period. Two further factors plausibly contribute: the target encodings had already turned the categorical signals into monotone numbers, leaving less for the trees to discover, and the moving base rate (7.95% → 5.53%) penalises flexible models more than constrained ones.

8.2 The feature that looked useless is the most important one

EVIDENCE 13 — permutation importance on validation

```
promised_days      0.1261
customer_state_te  0.0649
same_state         0.0436
distance_km        0.0255
main_seller_state_te 0.0202
...
promise_per_100km  0.0048
```

`promised_days` scored 0.487 on its own in the EDA — indistinguishable from noise. Inside the model it is the strongest feature by a wide margin, while `promise_per_100km`, the ratio the EDA built specifically to rescue it, is almost unused.

The explanation is that **univariate strength is not multivariate importance**. The promise's effect was masked by distance, with which it correlates 0.65. Once the model can see distance and promise as separate inputs, it can ask the right question itself — *given this distance, is this promise enough?* — and the effect appears. The hand-built ratio was necessary to see the phenomenon during analysis; it was not necessary to *represent* it inside a model that has both variables.

The remaining order matches the EDA exactly: customer state, then same-state, then distance. Geography governs the problem.

النسخة العربية

تقرير تسليم التاسك الثاني — نفس المحتوى وبالترتيب نفسه

٠. مطابقة معايير الإنجاز المذكورة في التاسك

المعيار كما ورد في التاسك	الحالة	الدليل
ست نوتبوكات، كل واحد يعمل شغلة واحدة، وتعمل بالترتيب من بداية نظيفة	مُنجز	القسم ١ — تُقِّد الستة من أولها لآخرها على نواة نظيفة، وأرقام التنفيذ متسلسلة
كل نوتبوك يحفظ artifacts ويقرأ artifacts الخطوة التي قبله	مُنجز	القسم ١ — سلسلة الـ artifacts، ١٣ ملفًا في <code>/artifacts</code>
تستطيع شرح سبب تقسيم البيانات بهذه الطريقة	مُنجز	القسم ٤ — التقسيم الزمني، والسبب رقم مقاس لا حجة نظرية
عندك أول نتيجة موديل مقارنة بـ baseline بسيط	مُنجز	القسم ٧ — أربعة خطوط أساس، ثم النموذج المختار، ثم نظرة واحدة إلى الـ test

١. البايبلاين

كل نوتبوك يعمل شغلة واحدة، يقرأ الـ artifact الذي كتبه ما قبله، ويكتب artifact خاصًا به لما بعده. لا يحتاج أي نوتبوك أن يكون سابقه قيد التشغيل. المخطط في القسم ١ من النسخة الإنكليزية.

الـ artifacts بصيغة **parquet** لا CSV، حتى تبقى أنواع الأعمدة — خصوصًا الطوابع الزمنية — محفوظة فلا يعيد النوتبوك التالي تحليلها.

٢. نوتبوك ١ — صف واحد لكل طلب

قاعدة هذه الخطوة في التاسك: *جمّع قبل أن تدمج*. ثلاثة جداول تحوي عدة صفوف للطلب الواحد، ودمجها مباشرة يضاعف الجدول بدل أن يصفه.

نتيجتان قادتا النوتبوك كله:

- **٧٧٥ طلبًا بلا صفوف في order_items** — الـ **INNER JOIN** كان سيبتلعها بصمت، بلا خطأ ولا تحذير. فكل دمج صار **LEFT JOIN** انطلاقًا من **orders**.
 - **geolocation** فيه **1,000,163** صف لـ **19,015** بادئة بريدية (~٥٣ نقطة لكل بادئة). جُمِّع إلى إحداثية وسيطة واحدة لكل بادئة قبل الدمج — الوسيط لا المتوسط لأن الجدول يحوي نقاطًا خارج البرازيل أصلًا.
- الطلبات متعددة البائعين (1,278 طلبًا):** عُرِّف **السطر الرئيسي** للطلب بأنه الأعلى سعرًا، ومنه يُؤخذ البائع والفئة والولاية والرمز البريدي — قاعدة ثابتة وقابلة للشرح — مع الاحتفاظ بـ **n_sellers** عمودًا مستقلًا حتى لا تضيع معلومة التعدد نفسها.
- كل دمج يمرّ بـ **validate="one_to_one"** الذي يرمي خطأ فورًا إن لم يكن المفتاح فريدًا — فيظهر فشل التجميع استثناءً لا صفوفًا مكررة.

٣. نوتبوك ٢ — الليل

التعريف المباشر هو **delivered > estimated**. وقبل قبوله يطرح النوتبوك سؤالًا واحدًا عن الموعد نفسه، فيبتين أن ١٠٠٪ من المواعيد الموعودة وقتها **00:00:00**.

الوعد إذن **يوم** لا لحظة، ومنتصف الليل غياب لمعلومة الساعة لا التزام بالتسليم عندها. فمقارنة الطوابع الزمنية تجعل طلبًا وصل في اليوم الموعود متأخرًا.

```
1,292 orders on which the two definitions disagree
= 16.5% of the positive class under the literal definition
50% 18.0 <- median arrival hour on the promised day
```

```
is_late (primary, calendar) : 6,534 late (6.77%)
is_late_ts (alternative, literal) : 7,826 late (8.11%)
```

القرار	السبب
الليل الأساسي هو القاعدة التقويمية (6.77%)	الوعد تاريخ. والطلبات الـ 1,292 المتنازع عليها تصل بوسيط ساعة 18:00 — تسمية تسليم السادسة مساءً في اليوم الموعد «تأخيرًا» تحقق ١٦.٥٪ حالات كاذبة في الفئة التي يتعلم منها النموذج.
التعريف الحرفي محفوظ في is_late_ts	العمودان معًا في الـ artifact، فتبديل التعريف سطر واحد.
المجتمع = delivered و له تاريخ تسليم	الطلب غير المسلم ليس متأخرًا ولا في الوقت. و٦ طلبات ملغاة تحمل تاريخ تسليم استبعدت: السؤال «هل يصل في الوقت» لا «هل وصل ثم ألغي».

وأعيد اشتقاق الليل بطريقة مستقلة تمامًا وقُورن صقًا صقًا: تطابق على 96,470 صقًا.

توازن الفئات: 6,534 متأخرًا من 96,470 — طلب متأخر مقابل ١٣.٨ في الوقت. عدم التوازن متوسط لا حاد: الأقلية نادرة نسبة لكنها كثيرة عددًا، فالصعوبة في اختيار المقياس لا في إيجاد الأمثلة.

٤. نوتبوك ٣ — لماذا التقسيم زمني

هذا هو المعيار الذي يطلب التاسك شرحه، والنوتبوك يجيب عنه بقياس لا بحجة.

الحجة الشائعة لا تنطبق هنا: يُرفض التقسيم العشوائي عادةً لأنه يسرّب تاريخ العميل عبر الحد. وقد فُحص أولًا فتبين أن ٩٧٪ من العملاء (90,549 من 93,350) طلبوا مرة واحدة — لا تاريخ عميل يسرّب أصلًا.

الدليل ٧ — الشهر مستعملًا كمُتنبئ

ROC-AUC using the month's late rate alone as the prediction: 0.698

معرفة الشهر وحده تعطي 0.698 — أعلى من النموذج الأساسي المقاس في التاسك الأول (0.679). التقسيم العشوائي يضع آذار ٢٠١٨ (١٩٪ تأخير) وحزيران ٢٠١٨ (١٪) في train و test معًا، فيحفظ النموذج معدلات الأساس الشهرية — وهي معلومة لا وجود لها في الإنتاج حيث يتنبأ لشهر لم يحدث.

القسم	النافذة	الطلبات	النسبة	التأخير
train	حتى 31-03-2018	64,320	66.7%	7.95%
validation	2018-04-01 ← 2018-05-31	13,547	14.0%	5.53%
test	من 01-06-2018	18,603	19.3%	3.61%

قُورنت ثلاث نقاط قطع مرشحة قبل الاختيار. نافذة validation أبكر كانت ستعطيها نسبة تأخير تقارب ثلاثة أضعاف نسبة الـ test، فتصير مؤشرًا مضرًا على الشيء الذي وُجدت لتتنبأ به. الحدود المختارة تُبقي الـ validation والـ test متجاورين زمنيًا.

اختلاف نسب التأخير مقصود. التاسك يطلب الحفاظ على نسبة الليل بين الأقسام / إذا كان التقسيم عشوائيًا؛ وهذا التقسيم زمني، فتركت النسب كما هي. الانتقال من 7.95٪ إلى 3.61٪ هو الظاهرة التي على النموذج مواجهتها بصدق لا عيبًا يُصحح.

حدّ مفصّل عنه: التقسيم بتاريخ الشراء بينما الليل لا يُعرف إلا عند التسليم. 3,852 طلبًا في الـ train (٥.٩٩٪) سلّمت بعد فتح نافذة الـ validation. العلاج الصارم (embargo) يكلف ٦٪ من بيانات التدريب مقابل ربح نظري صغير، فأُبقيت الصفوف وأُعلن القيد بدل إخفائه.

٥. نوتبوك ٤ — ما قرّره ال EDA

نُفِّذ على `train.parquet_03` وحده. الهدف لم يكن الرسوم بل قرارين مكتوبين: أي فيتشرز نبني، وأي عائلة نماذج تناسب شكل البيانات.

الوعد يبدو عديم الفائدة وليس كذلك: `promised_days` أعطى $AUC = 0.487$ وحده. لكن AUC يقيس علاقة رتيبة، والشكل الفعلي لا مقلوب: الوعود القصيرة آمنة (٤.٩٪) والطويلة آمنة (٤.٧٪) والخطر في الوسط (١٠.٤٪).

الارتباط الذي يفسّر ذلك: ارتباط الوعد بالمسافة 0.65 — الشركة تُطيل وعدها مع البُعد، فيلغي الأثران بعضهما عند النظر إلى الوعد منفردًا. والكمية المفيدة إذن ليست كم يومًا وُعد بل كم يومًا وُعد لكل ١٠٠ كم، أي هل الوعد سخّي أم طموح نسبةً إلى الشحنة. ونتيجتها $AUC = 0.612$ — أقوى إشارة في البيانات.

النتيجة	القياس
المسافة تتصدّر الأعمدة الخام	$AUC = 0.589$ ، رتيبة من ٣.٧٪ إلى ١٢.٨٪ عبر العشرات
نفس الولاية تنصّف الخطر	4.61% مقابل 9.66%
الولاية تفرق كثيرًا	4.72% (PR) إلى 19.43% (MA)
نقص الإحداثيات إشارة بذاته	١٧٣ طلبًا بلا مطابقة جغرافية تتأخر بنسبة 10.98% مقابل 7.94% — بادئة بريدية غائبة عن الجدول المرجعي تعني منطقة نائية. أُبقيت كعمود مؤثّر بدل أن تُملأ وتُدفن
خصائص المنتج لا تحمل شيئًا	السعر والوزن والحجم وعدد العناصر كلها بين 0.49 و 0.53 — التأخير مشكلة مسار لا مشكلة طرد
يوم الأسبوع والساعة مسطّحان	7.38% إلى 8.55% عبر أيام الأسبوع
الشهر قوي لكنه مستبعد	0.75% إلى 18.96%، لكنه $آذر/٢٠١٨$ تحديدًا لا «آذار» — لا يُعمّم على المستقبل

٦. نوتبوك ٥ — الفيتشرز والكائنات التي يجب حفظها

٣١ ميزة. والانضباط الذي تفرضه هذه الخطوة: كل إحصاء يُتعلّم من قسم التدريب وحده، وكل ما يُتعلّم يُحفظ. في الإنتاج يصل طلب واحد — ولا يمكن حساب وسيط منه.

الكائن المحفوظ	ماذا تعلّم	ماذا يكسر إعادة تدريبه في الإنتاج
<code>kept_categories</code>	أي فئات المنتجات ال ٧٢ ليست نادرة (٤٤ محتفّظًا بها، ٢٨ مجمّعة)	فئات مختلفة تُجمّع فتتغيّر معاني الأعمدة
<code>target_encoders</code>	نسبة التأخير المنعّمة لكل ولاية وفئة	ترميز مبني على بيانات لم يرها النموذج
<code>imputer</code>	وسيط كل ميزة	فراغات تُملأ بأرقام تخالف ما جرى في التدريب
<code>onehot_payment</code>	قيم نوع الدفع وترتيبها	ترتيب أعمدة مختلف يبعثر الميزات
<code>scaler</code>	متوسط وانحراف كل ميزة	نموذج خطي يقرأ مدخلات بلا معنى

الترميز المستهدف بلا تسريب للبيانات: استبدال الولاية بنسبة تأخيرها التاريخية يخلق خطرين، وكلاهما عولج صراحةً:

- **الفئات الصغيرة تنتج ضجيجًا.** ولاية فيها ١٢ طلبًا منها ٢ متأخران تقرأ ١٦.٧٪. التنعيم يسحب كل تقدير نحو المعدل العام بقوة ٥٠ مشاهدة وهمية، فتكاد الفئات الصغيرة لا تتحرّك عن المعدل بينما تحتفظ الكبيرة بنسبتها.
- **الصفوف كانت ستري لبيها.** ترميز صف تدريبي بمتوسط محسوب من مجموعة التدريب يُدخل لبيل ذلك الصف في ميزته. لذلك يُحسب الترميز داخل ال `train` خارج الطيّة — خمس طيّات، كل واحدة تُرمّز من الأربع الأخرى. أما ال `validation` وال `test` فيستعملان خريطة كامل ال `train`، وهو ما سيستعمله الإنتاج تمامًا.

OK 31 features, no leakage, no gaps, identical structure across splits
 OK rebuilding from the loaded objects matches the notebook output exactly
 -> the pipeline is genuinely reproducible, not just claimed to be

الخلية الأخيرة تعيد تحميل transformers.joblib_05 من القرص، وتبني طلبًا واحدًا من الـ test عبر الكائنات المحملة وحدها، وتؤكد تطابق الناتج مع ما أنتجه النوتبوك. وهذا هو شرط التاسك بأن تطابق مخرجات بايبلين الإنتاج مخرجات النوتبوك — مُختبرًا لا مفترضًا.

قائمة الحظر متغيّر في الكود لا تعليق، وتأكيد (assert) يفشل إن وصل أي عمود محظور إلى جدول الفيتشرز. والـ scaler مدرّب ومحفوظ لكنه غير مطبّق عمدًا: الأشجار لا تحتاجه، والجداول المحفوظة تبقى قابلة للقراءة.

٧. نوتبوك ٦ — النتائج

لا يُحمّل ملف الـ test إلا في القسم الأخير، بعد تثبيت كل قرار.

لماذا الدقة ليست المقياس: نموذج يتنبأ بأن لا شيء يتأخر يحقق **94.47%** دقة ويمسك **صفر** من ٧٤٩ طلبًا متأخرًا. المقاييس المستعملة بدلًا منها: PR-AUC (أساسي)، وROC-AUC (قابل للمقارنة عبر أقسام تختلف معدلات أساسها)، و**recall عند ميزانية إنذار ثابتة** — كم متأخرًا نمسك لو استطاع الفريق التدخّل في أخطر ٥%.

النموذج	ROC-AUC	PR-AUC
promise_per_100km — B1 وحدها، بلا تدريب إطلاقًا	0.6953	0.1433
M1 — Random Forest	0.7303	0.1490
M2 — Gradient Boosting (عمق ٢)	0.7526	0.1709
الانحدار اللوجستي — المختار	0.7739	0.1709

المقياس	validation	test
الطلبات	13,547	18,603
نسبة التأخير	5.53%	3.61%
ROC-AUC	0.7739	0.7107
PR-AUC	0.1709	0.0849
recall عند ميزانية ٥%	20.7%	15.0%
lift عند ٥%	4.14x	3.01x

تشغيليًا: التدخّل في أخطر ٥% من الطلبات يمسك ١٥% من المتأخرة على بيانات لم يرها النموذج إطلاقًا — ثلاثة أضعاف ما يجده الاستهداف العشوائي. والرقم المرجعي من التاسك الأول (ميزات لحظة الشراء، بلييل وتقسيم مختلفين) كان ROC-AUC = 0.679. **عن الفجوة بين 0.774 و 0.711:** ROC-AUC مقياس ترتيبى لا يتأثر بمعدل الأساس، فهذا هبوط حقيقي في التعميم لا أثر حسابي. التدريب ينتهي في آذار ٢٠١٨ ونافذة الاختبار تمتدّ من حزيران إلى آب — أي أن النموذج يتنبأ لفترة تبعد ثلاثة إلى خمسة أشهر عن آخر ما رآه. والاستنتاج التشغيلي إعادة تدريب دورية لا نشر لمرة واحدة.

٨. نتيجتان مقاستان خالفتا التوقع

٨.١ الأشجار خسرت

توقع نوتبوك ٤ فوز نماذج الأشجار لثلاثة أسباب مذكورة: منحى الوعد غير الرتيب، والالتواء الشديد، والتفاعلات الحقيقية بين الأعمدة. وتفوق عليها الانحدار اللوجستي في المقياسين. والدليل على السبب ظاهر في الضبط نفسه — أداء التعزيز يتراجع كلما زادت الجولات مع ثبات كل شيء آخر.

هذا ليس نقص تدريب بل حفظ للفترة التدريبية. وعاملان آخران يُرجّح إسهامهما: الترميز المستهدف كان قد حوّل الإشارات التصنيفية إلى أرقام رتيبة فلم يبق للأشجار كثير لتكتشفه، ومعدل الأساس المتحرّك (7.95% ← 5.53%) يعاقب النماذج المرنة أكثر من المقيدة.

٨.٢ الميزة التي بدت عديمة الفائدة هي الأهم

الدليل ١٣ — permutation importance على ال validation

```
promised_days      0.1261
customer_state_te   0.0649
same_state          0.0436
distance_km         0.0255
main_seller_state_te 0.0202
...
promise_per_100km  0.0048
```

`promised_days` سجّل 0.487 وحده في ال EDA — لا يميّز عن الضجيج. وداخل النموذج هو أقوى ميزة بفارق واسع، بينما `promise_per_100km` — النسبة التي بناها ال EDA خصيصًا لإنقاذه — تكاد لا تُستعمل.

والتفسير أن **القوة أحادية المتغير ليست الأهمية متعددة المتغيرات**. أثر الوعد كان مُقنّعًا بأثر المسافة التي يرتبط بها 0.65. وحين يرى النموذج المسافة والوعد مدخلين منفصلين يستطيع أن يسأل السؤال الصحيح بنفسه — *بالنظر إلى هذه المسافة، هل هذا الوعد كافٍ؟* — فيظهر الأثر. النسبة المبنية يدويًا كانت ضرورية لرؤية الظاهرة أثناء التحليل، لا لتمثيلها داخل نموذج يملك المتغيرين.

وبقية الترتيب يطابق ال EDA تمامًا: ولاية الزبون، ثم نفس الولاية، ثم المسافة. الجغرافيا تحكم المشكلة.